

AI OPERATING PLATFORM

Especificación Arquitectónica, Mapa Conceptual Hexagonal y Flujo Operacional de Extremo a Extremo (Lifecycle)

PRINCIPIOS ARQUITECTÓNICOS FUNDACIONALES

Arquitectura Hexagonal Pura (Ports & Adapters)

El núcleo de dominio nunca conoce HTTP, frameworks web ni SDKs propietarios de proveedores.

Zero Runtime Dependencies

100% construido con primitivas estándar de Node.js. Cero librerías npm en tiempo de ejecución.

Gobernanza Fail-Closed Centralizada

Toda llamada a modelo, tool o agente es evaluada por políticas. Ante fallo o duda, se bloquea por defecto.

Trace Correlation Ubicua

Cada tarea, ejecución y observación de auditoría comparte un traceId inmutable de extremo a extremo.

§ Índice General de Contenidos

Esta publicación técnica documenta de manera exhaustiva la arquitectura y operativa de la **AI Operating Platform**, desde sus principios conceptuales hasta el ciclo de vida de ejecución en código.

1	Visión y Filosofía del Sistema Operativo de IA	— Diferenciación enterprise, anti-patrones y propósito	Pág. 3
2	Arquitectura Hexagonal y Mapa Conceptual por Capas	— Capas 4 a 0 y diagrama de dependencias	Pág. 3
3	La Tríada Fundamental: Agent vs Task vs Execution	— Invariante cardinal y tabla comparativa	Pág. 4
4	Los 4 Pilares de Control, Resiliencia y Gobernanza	— Trazabilidad, Fail-Closed, Memoria e Inmutabilidad	Pág. 4
5	Flujo Operacional de Extremo a Extremo (Execution Lifecycle)	— Diagrama de secuencia y los 8 pasos	Pág. 5
6	Frontera de Plataforma: Platform API & Web Control Plane	— Endpoints REST, seguridad y SPA	Pág. 6
7	Evolución de la Hoja de Ruta Tecnológica (Roadmap v0.1 – v1.0)	— Hitos pasados y futuro autónomo	Pág. 6

i Resumen Ejecutivo de la Plataforma

La **AI Operating Platform** aborda la fragilidad de las arquitecturas tradicionales basadas en wrappers descontrolados de LLMs. Mediante una arquitectura hexagonal estricta, la plataforma proporciona un motor determinista, auditable y seguro donde los modelos y herramientas son capacidades explícitas gobernadas centralmente.

Propósito Estratégico: Permitir que aplicaciones empresariales (como *AI Commerce* o sistemas ERP) consuman agentes y flujos orquestados mediante contratos de API estables, sin acoplarse jamás al proveedor de modelos ni a la infraestructura de almacenamiento subyacente.

1 Visión y Filosofía de la Plataforma

El proyecto **ai-operating-platform** no es una aplicación de chat, ni un simple script con llamadas a APIs de LLM. Es la construcción progresiva de una **infraestructura de sistema operativo empresarial** diseñada para desacoplar modelos, herramientas, memoria, orquestación, agentes y políticas de seguridad.

LO QUE NO ES ESTA PLATAFORMA

- **NO es un Chatbot:** No es una interfaz conversacional opaca.
- **NO es un Wrapper de Proveedor:** Agnóstico de SDKs propietarios.
- **NO depende de librerías externas:** Cero paquetes runtime npm.
- **NO tiene cajas negras autónomas:** Sin bucles descontrolados.

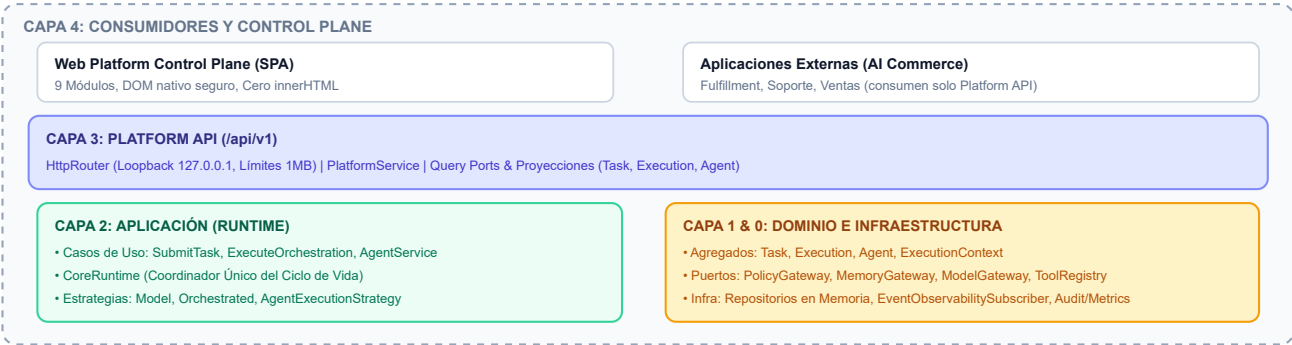
LO QUE SÍ ES LA PLATAFORMA

- **Un AI Operating Engine:** Motor unificado que ejecuta tareas y orquestaciones.
- **Gobernanza Fail-Closed:** Ante duda o falla de política, bloquea por defecto.
- **Trace Correlation Ubicua:** Cada acción está vinculada a un traceId inmutable.
- **Hexagonal y Modular:** Reemplazo de adaptadores sin tocar el dominio.

2 Mapa Conceptual: Arquitectura Hexagonal por Capas

La plataforma organiza sus componentes en cuatro capas concéntricas donde el **Dominio** ocupa el centro soberano y las dependencias siempre apuntan hacia adentro:

DIAGRAMA DE CAPAS Y DEPENDENCIAS DE LA PLATAFORMA



3 La Tríada Fundamental: Agent vs Task vs Execution

INVARIANTE CARDINAL: AGENT DOES NOT REPLACE EXECUTION.

Un Agente es un descriptor de configuración y capacidades. Jamás ejecuta código de manera aislada ni posee un ciclo de vida independiente. Toda acción de un Agente se ejecuta a través de la secuencia estándar: `SubmitTask` → `CoreRuntime` → `Execution`.

Dimensión	Agent (Quién)	Task (Qué)	Execution (Cómo)
Definición	Perfil de identidad, instrucciones fijas, modelo asociado, tools autorizadas y scope de memoria.	Unidad duradera de trabajo o intención solicitada a la plataforma.	Intento técnico atómico de ejecutar una tarea bajo un contexto y estrategia específicos.
Capa	Agregado de Dominio / Configuración declarativa.	Agregado de Dominio / Ciclo de Vida Operacional.	Agregado de Dominio / Runtime atómico.
Máquina Estados	ACTIVE ↔ INACTIVE (Con versionado monotónico).	QUEUED → RUNNING → COMPLETED / FAILED / CANCELLED	PENDING → RUNNING → SUCCEEDED / FAILED / CANCELLED
Gobernanza	Posee una lista blanca (whitelist) estricta de tools (<code>agent.tools</code>).	Verifica políticas en frontera de submisión.	Evalúa <code>PolicyGateway</code> (Fail-Closed) inmediatamente antes de llamar a modelos/tools.
Persistencia	<code>AgentRegistry</code> (In-Memory).	<code>TaskRepository</code> (Durable, retiene inputs y outputs).	<code>ExecutionRepository</code> (Rastrea latencias, traza, errores).

4 Los 4 Pilares de Control, Resiliencia y Gobernanza

1. Trace Correlation Ubicua

Cada solicitud adopta un `traceId` (UUIDv4) inmutable. Todo evento emitido, registro en `AuditLog` y métrica comparte la misma traza de principio a fin.

2. Gobernanza Fail-Closed

Antes de invocar modelos, herramientas o agentes, se evalúa `PolicyGateway`. Si la política deniega o falla, el sistema **bloquea por defecto**.

3. Aislamiento Estricto de Memoria

Cada agente opera bajo un `memoryScope` delimitado (`agent-${id}`). Un agente no puede acceder a claves de memoria de otro agente.

4. Inmutabilidad de Contexto

El `ExecutionContext` encapsula identidades y parámetros de ejecución como objetos inmutables congelados con `Object.freeze()`.

5 Flujo Operacional de Extremo a Extremo (Execution Lifecycle)

El siguiente diagrama y desglose secuencial ilustra con exactitud qué ocurre en la plataforma desde que un cliente externo invoca la API hasta que la tarea concluye y es persistida:



- 1 Recepción y Sanitización en Platform API:** El cliente emite POST /api/v1/agents/:id/executions. El HttpRouter valida Content-Type (JSON), tamaño de carga (<1MB) e ID normalizado.
- 2 Verificación de Perfil de Agente:** AgentService consulta AgentRegistry, validando que el agente exista y su estado sea ACTIVE. Agentes inactivos son rechazados de inmediato con HTTP 400.
- 3 Creación de Tarea (Task Lifecycle):** Se invoca el caso de uso SubmitTask, que crea una Task en estado QUEUED con un traceId inmutable y emite el evento task.created.
- 4 Activación en CoreRuntime:** El motor central transiciona la Tarea a RUNNING, crea una Execution (PENDING → RUNNING) y delega a AgentExecutionStrategy.
- 5 Gobernanza Fail-Closed Obligatoria:** La estrategia evalúa PolicyGateway.evaluate(). Si la política deniega o hay fallas en la evaluación, la ejecución se interrumpe y falla cerrado.
- 6 Aislamiento de Memoria y Autorización de Herramientas:** La estrategia vincula el memoryScope del agente y valida que cualquier herramienta invocada esté en su lista blanca (agent.tools).
- 7 Inferencia Agnóstica de Modelo:** Se combinan las instrucciones declarativas del agente con la entrada de la tarea y se invoca el ModelGateway.
- 8 Persistencia, Observabilidad y Respuesta:** CoreRuntime marca la Ejecución como SUCCEEDED y la Tarea como COMPLETED. EventObservabilitySubscriber vuelca la traza a AuditLog e incrementa métricas. La API responde HTTP 200 con el DTO completo.

6 Frontera de Plataforma: Platform API & Web Control Plane

La **Platform API** define el límite estricto entre el motor interno y el mundo exterior. Los clientes consumen exclusivamente contratos DTO a través de endpoints REST seguros:

Método	Ruta REST (/api/v1)	Descripción y Garantía Operacional
GET	/api/v1/status	Salud del sistema, versión activa (0.8.0), uptime y conteo de recursos.
GET, POST	/api/v1/agents	Lista agentes registrados o registra uno nuevo con validación de modelo y tools.
GET, PUT	/api/v1/agents/:id	Consulta configuración o actualiza propiedades incrementando el versionado.
POST	/api/v1/agents/:id/activate	Habilita el agente para permitir que sea invocado por clientes.
POST	/api/v1/agents/:id/executions	Ejecuta el agente atravesando CoreRuntime de forma gobernada.
POST	/api/v1/executions	Submisión de tareas generales mediante el caso de uso SubmitTask.
POST	/api/v1/orchestrate	Despacho de secuencias multi-operación a través de CoreRuntime.
GET	/api/v1/audit, /metrics	Flujo estructurado de observaciones y contadores numéricos en tiempo real.

7 Evolución de la Hoja de Ruta (Roadmap v0.1 – v1.0)

Versión	Enfoque y Alcance	Criterio de Salida	Estado
v0.1 – v0.3	Fundamentos, Task System, CoreRuntime, Models agnósticos y Tools seguras.	Transiciones verificadas y ejecución observable.	✓ Completed
v0.4 – v0.6	Orquestación secuencial, Contexto inmutable, Memoria, Gobernanza y Observabilidad.	Políticas fail-closed y timeline de auditoría correlacionado.	✓ Completed
v0.7	Platform API (/api/v1) y Web Platform Control Plane (SPA nativa sin dependencias).	SPA desacoplada del motor interno, seguridad loopback.	✓ Completed
v0.8 (Actual)	Capacidad formal de Agents, Whitelist de Tools, Scopes de Memoria, AgentExecutionStrategy.	Agentes gobernados ejecutándose a través de CoreRuntime.	✓ Completed
v0.9	Operaciones autónomas acotadas, bucles de reflexión y delegación multi-agente.	Límites operativos estrictos y auditoría de bucles.	📄 Siguiente
v1.0	Enterprise AI Operating Platform integrada con soporte Multi-Tenant y nube.	Plataforma enterprise para producción.	🕒 Planeado